

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

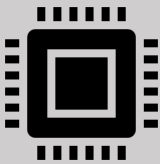
División de Ciencias de la ingeniería

Ingeniería

Estructura de Datos Sección “A”

Ing. Mario Moises Ramirez Tobar

Primer Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Análisis de complejidad Práctica#1



Descripción:

En el siguiente documento se da a conocer una breve explicación sobre la complejidad y la estrategia utilizada para barajar las cartas. Utilizando el método Fisher-Yates. Que es un método clásico que permite barajar sin mayor complejidad al igual que dar a conocer el análisis de la búsqueda de valores en una lista de cartas.

Análisis de complejidad Lista Enlazada

En esta práctica se utilizó una lista doblemente enlazada. Tal como fue vista en clase esta al tener un doble enlace es consciente de su nodo anterior y del siguiente. Por lo tanto permite insertar objetos o eliminar objetos en cualquier posición ya que cada **Nodo** apunta hacia su nodo anterior y siguiente.

Donde cada nodo contiene:

- Un dato genérico **T**
- Un puntero al siguiente nodo
- Un puntero al nodo anterior

Su complejidad pese a que esta pueda tener **n** número de elementos su complejidad no depende de su tamaño por lo tanto se dice que:

- Crear nodo -> $O(1)$
- Ajustar punteros -> $O(1)$
- Actualizar head -> $O(1)$
- Guardar nodo -> $O(1)$

Sin embargo a generar una búsqueda de cierto nodo en un índice su complejidad puede variar ya que se tiene que recorrer nodo por nodo pero solo es en el peor de los casos esta puede llegar a una complejidad de:

$O(n)$

De igual forma aplica el mismo factor al momento de eliminar ya que hay que buscar el nodo y ajustar el puntero. Entonces se puede decir que también su complejidad sería:

$O(n)$

Cosa que para esta práctica fue muy utilizada constantemente ya que se requería sacar cartas desde donde el usuario indicara.

Análisis de complejidad Barajado

El proceso de barajado de cartas dentro de la clase **Partida** tiene como objetivo mezclar aleatoriamente los elementos almacenados en una estructura **ListaEnlazada<Carta>**. La idea nace principalmente de que el simple hecho de barajar dentro de una lista enlazada no es lo más práctico ya que está de por sí posee una complejidad mencionada en la página anterior. Donde al querer hablar desde la lista enlazada su complejidad aumentaría a **$O(n^2)$** .

Esto significa que el tiempo de ejecución crece proporcionalmente al número de cartas, bajo este panorama no era factible barajar usando la lista. Entonces se optó por eliminar datos de la lista, meterlos a un arreglo del mismo tipo que la lista enlazada y desde ese arreglo al tener una complejidad **$O(1)$** . El método de barajado iba a ser más eficiente llegando a una complejidad temporal lineal de:

$$O(n)$$

De esta forma se redujo drásticamente de una posible **$O(n^2)$** a una **$O(n)$** .

Este proceso de barajado es simple. Consiste en recorrer el arreglo de cartas acorde al tamaño que este tenga e ir generando índices random dentro de los índices del arreglo e ir intercambiando los datos cada vez que se suelta un random. Por lo tanto al ir recorriendo el arreglo de ir intercambiando constantemente se consigue un método de barajado óptimo y bastante rápido.

El algoritmo implementado para barajar:

- Es óptimo en complejidad temporal
- Es matemáticamente uniforme
- Escala linealmente
- Es adecuado para el dominio del problema

Visualización del procedimiento de barajado:



Esto nos da una guía de cómo es que funciona. Que simplemente al terminar de hablar y mover todo dentro de un arreglo. Se vuelve a pasar todo a la lista.

Importante:

Para aumentar el índice de aleatoriedad lo que se hace es que al armar la lista se utiliza la inserción solo al frente de la lista enlazada. Y luego al momento de volver a insertar todo a la lista se inserta todo por la parte de atrás. así generando todavía más desorden dentro de la lista por si en dado caso hubieron algunas cartas que quedaron contiguas.